4 Queens using Hill Climbing Algorithm

Code:

```
import random

N = 4

def cost(board):
    """Count pairs of queens attacking each other."""
    conflicts = 0
    for i in range(N):
        for j in range(i + 1, N):
            if (board[i] == board[j] or
                abs(board[i] - board[j]) == abs(i - j)):
                conflicts += 1
    return conflicts


def get_neighbors(board):
    """Move one queen to another row in its column (0-indexed: rows 0 to N-1)."""
    neighbors = []
    for col in range(N):
        for row in range(N):
            if row != board[col]:
                neighbor = board.copy()
                neighbor[col] = row
                neighbors.append((col, row, neighbor))
    return neighbors


def print_grid(board):
    """Print the N-Queens solution as an N x N grid."""
    print("\nGrid Representation:")
    for row in range(N):
        line = ["Q" if board[col] == row else "." for col in range(N)]
        print(" ".join(line))


def hill_climbing(initial_state=None):
    restart = 0

    while True:
        restart += 1
        # Use initial_state for the first attempt, then random boards for restarts
        if restart == 1 and initial_state is not None:
            current = initial_state.copy()
        else:
            current = [random.randint(0, N - 1) for _ in range(N)]

        print(f"\nRestart {restart}: Initial state = {current}")
        print("Initial cost =", cost(current))

        while True:
```

```
            current_cost = cost(current)

            if current_cost == 0:
                print("\nGOAL FOUND!")
                print("Final board:", current)
                print("Final cost:", current_cost)
                print_grid(current)
                return current

            neighbors = get_neighbors(current)

            print("\nCurrent state:", current)
            print("Current cost:", current_cost)
            print("Neighbor states and costs:")

            for col, row, neighbor in neighbors:
                print(
                    f" Move column {col} to row {row}: "
                    f"{neighbor}, cost = {cost(neighbor)}"
                )

            # Choose a minimum-cost neighbor.
            best_cost = min(cost(nb) for _, _, nb in neighbors)
            best_neighbors = [
                nb for _, _, nb in neighbors
                if cost(nb) == best_cost
            ]
            best = random.choice(best_neighbors)

            # Restart if no improvement is possible.
            if best_cost >= current_cost:
                print("Local minimum reached; restarting...")
                break

            current = best


if __name__ == "__main__":
    hill_climbing(initial_state=[3, 1, 2, 0])
```

Output:

```
PS C:\Users\Administrator\Desktop\1BM24CS316> & c:\python314\python.exe c:/Users/Administrator/Desktop/1BM24CS316/HillClimbing4Queen.py

Restart 1: Initial state = [3, 1, 2, 0]
Initial cost = 2

Current state: [3, 1, 2, 0]
Current cost: 2
Neighbor states and costs:
 Move column 0 to row 0: [0, 1, 2, 0], cost = 4
 Move column 0 to row 1: [1, 1, 2, 0], cost = 2
 Move column 0 to row 2: [2, 1, 2, 0], cost = 3
 Move column 1 to row 0: [3, 0, 2, 0], cost = 2
 Move column 1 to row 2: [3, 2, 2, 0], cost = 4
 Move column 1 to row 3: [3, 3, 2, 0], cost = 3
 Move column 2 to row 0: [3, 1, 0, 0], cost = 3
 Move column 2 to row 1: [3, 1, 1, 0], cost = 4
 Move column 2 to row 3: [3, 1, 3, 0], cost = 2
 Move column 3 to row 1: [3, 1, 2, 1], cost = 3
 Move column 3 to row 2: [3, 1, 2, 2], cost = 2
 Move column 3 to row 3: [3, 1, 2, 3], cost = 4
Local minimum reached; restarting...

Restart 2: Initial state = [3, 3, 2, 2]
Initial cost = 3

Current state: [3, 3, 2, 2]
Current cost: 3
Neighbor states and costs:
 Move column 0 to row 0: [0, 3, 2, 2], cost = 3
 Move column 0 to row 1: [1, 3, 2, 2], cost = 2
 Move column 0 to row 2: [2, 3, 2, 2], cost = 5
 Move column 1 to row 0: [3, 0, 2, 2], cost = 2
 Move column 1 to row 1: [3, 1, 2, 2], cost = 2
 Move column 1 to row 2: [3, 2, 2, 2], cost = 4
 Move column 2 to row 0: [3, 3, 0, 2], cost = 1
 Move column 2 to row 1: [3, 3, 1, 2], cost = 3
 Move column 2 to row 3: [3, 3, 3, 2], cost = 4
 Move column 3 to row 0: [3, 3, 2, 0], cost = 3
 Move column 3 to row 1: [3, 3, 2, 1], cost = 4
 Move column 3 to row 3: [3, 3, 2, 3], cost = 5

Current state: [3, 3, 0, 2]
Current cost: 1
Neighbor states and costs:
 Move column 0 to row 0: [0, 3, 0, 2], cost = 1
 Move column 0 to row 1: [1, 3, 0, 2], cost = 0
 Move column 0 to row 2: [2, 3, 0, 2], cost = 3
 Move column 1 to row 0: [3, 0, 0, 2], cost = 2
 Move column 1 to row 1: [3, 1, 0, 2], cost = 1
 Move column 1 to row 2: [3, 2, 0, 2], cost = 2
 Move column 2 to row 1: [3, 3, 1, 2], cost = 3
 Move column 2 to row 2: [3, 3, 2, 2], cost = 3
 Move column 2 to row 3: [3, 3, 3, 2], cost = 4
 Move column 3 to row 0: [3, 3, 0, 0], cost = 3
 Move column 3 to row 1: [3, 3, 0, 1], cost = 3
 Move column 3 to row 3: [3, 3, 0, 3], cost = 3

GOAL FOUND!
Final board: [1, 3, 0, 2]
Final cost: 0

Grid Representation:
. . Q .
Q . . .
. . . Q
. Q . .
PS C:\Users\Administrator\Desktop\1BM24CS316>
```